

On the Strategic Importance of Programmable Middleboxes

Thomas Fuhrmann

Institut für Telematik, Universität Karlsruhe (TH), Germany

Abstract—Network protocols suffer from a lock dictated by the need for standardization and Metcalf’s law. Programmable middleboxes can help to relieve the effects of that lock. This paper gives game theoretic arguments that show how the option of having middleboxes can raise the quality of communication protocols. Based on this analysis, design considerations for active and programmable networks are discussed.

I. INTRODUCTION

Although middleboxes often spoil the concept of end-to-end communication in the Internet, middleboxes, like network address translators, firewalls, proxies, etc., have become an everyday occurrence: network address translators, firewalls, proxies, and many more. Often, an economic need drives the development of such devices: saving valuable IP-addresses, circumventing the need to secure individual applications against attacks, etc. In many cases, such middleboxes cause significant turbulence in their wake that has to be overcome by additional engineering approaches not always on the straight line of ideal concepts [2]. Application dependent functionality implemented in the middleboxes is among these approaches to cure such adverse reactions.

Recent activities aim at pushing back this application dependence from the middleboxes back to the edge of the network where it is thought to be handled more easily. [4] and [5], e.g., discuss a framework and protocol that pursue that goal. Most interestingly, this approach is in contrast to the concept of active and programmable networks that aim at shifting application dependent functionality “on the fly” onto the inner network nodes. E.g., [1] gives a taxonomy of middleboxes that entirely omits active and programmable networks.

This paper tries to shed some light onto that conflict by considering the strategic implications of the various approaches (pure end-to-end network, middleboxes, programmable network nodes). Keeping in mind the different objectives of the process’ stakeholders, these considerations might be able to provide some guidance in the

challenge to predict the future directions in active and programmable networking.

This paper *does not* aim at providing answers to technical questions nor does it state facts that could be used to judge one concrete solution or the other. Following the ideas of CLARK et al. it rather raises questions that could be worth considering when one has to weigh between two, perhaps otherwise equivalent, engineering approaches in the field of active and programmable networks.

II. THE LOCKING GAME

The arguments presented in the following use terms lent from the field of game theory [3]. There, the *strategic options* are explored that *players* can take during a *game*. A game is hence merely a collection of rules that describe how the players may interact. Games can be classified according to whether the players form jointly acting groups or not, whether the players may base their decision on the other players’ decisions or, conversely, must decide independently, and whether the players have or have not information about the other players’ actions.

We use this mind set for our analysis even though this paper does not aim at giving a full mathematical analysis of the questions raised here.

Originally, the Internet was devised to provide connectivity for network applications located in the end-devices. This created two well-separated areas for the game: the network and the applications. Both of these areas can and should be analyzed separately since they follow different rules. The former is exposed to the strong need of having *one* standard set of protocols. The latter is open to a multitude of different outcomes, i.e. players are able to develop competing solutions according to the market demands. In principle, many such solutions may co-exists.

However, the often cited effect of Metcalf’s law, that a network’s utility increases exponentially with the number of nodes, drives users to *get locked* into one protocol no matter if the instances using this protocol are situated inside the network or at its edge. In many cases, this

is achieved only indirectly through the use of a common data format rather than directly by an actual application layer protocol. E.g., if a vendor uses a communication protocol C_1 to exchange data of format F_1 and other vendors use C_k and F_k accordingly, experience shows that the combination C_i and F_j with $i \neq j$ is unlikely. In other words, the game is cooperative. Players group into explicit “strategic partnerships” bundling the different layers of communication protocols and network-related data formats — or players decided to follow the crowd and be compatible to de-facto standards.

As a result of this locking, the distinction of the originally well-separated areas for the game vanishes: Any network exhibits a *strong* driving force that pushes the players into using only one solution for each given problem. Clearly, the rules differ between the two areas, but the resulting structure is the same.

This might be surprising at first sight, since unlike the network layer, the application layer is not always subject to formal, and more importantly open, standardization. Many application layer protocols have been standardized by the IETF, but a significant number has not. Originally, this was thought to create a multitude of applications each of which is best suited for a specific purpose. But in fact, this gives the owners of the de-facto standards considerable power over the future directions of network engineering. In other words, there is a *large* reward for creating a successful solution to a given problem (or market demand). But at the same time, the nature of that reward can reduce the number of players in that game to one. Often, the former is considered to be good and desirable (free competition creating benefit for society) while the latter (a monopoly) is considered to be bad and to be avoided. Please note, that we do not imply any of the two judgments with our analysis but rather describe the structure of the game.

Based on this observation, we can now analyze the effect of this game, more specifically we address the question to which level of success a protocol needs to be developed to create such a lock.

Let Q_i be a linear measure of the quality of a protocol, i.e. how well it is suited for the respective purpose, how many resources it occupies, how much disturbance it creates in the rest of the system. According to METCALF the overall benefit that results from using a protocol i is then $Q_i e^{N_i}$ where N_i is the number of deployed instances of that protocol. If we ideally assume perfect information in the game, i.e. if we assume that each newly deployed instance uses the protocol with the largest benefit, Q^* the *entry barrier* (the quality a new player must

achieve) rises like $Q^* = \max Q_i e^{N_i}$ where $\sum_i N_i$, the total number of deployed instances, increases exponentially with time. (According to the properties of exponential growth the same holds for each of the individual protocols, i.e. a protocol i that is introduced at time t_i follows an increase according to $N_i \propto e^{\lambda(t-t_i)}$ where λ is the growth parameter.)

From this we can conclude: The later a protocol is introduced the larger is its entry barrier. The growth of the barrier Q^* follows

$$Q^* \propto e^{e^{\lambda \Delta t}} \quad (1)$$

where Δt is the time since the establishment of a competing protocol.

This very strong rise of the of the entry barrier leads players into quickly releasing a protocol. As soon as a protocol is accepted by a small number of users, any substitute protocol would have to be overwhelmingly better than the existing protocol. No matter how ill-suited a protocol is, once it provides a marginal benefit for the early adopters it starts being used and will consequently quickly reach a state that prevents better-suited solutions from taking over. This is even more remarkable since due to the exponential growth of the number of instances the overall benefit (the sum over all instances) from a given improvement rises exponentially with time, too.

So far, we have considered distinct protocols, i.e. protocols that are incompatible. Compatible protocols slightly change the game, in that such a protocol can benefit from the installed base of instances of all the protocols it is compatible to. Yet, we have to assume that with those instances a protocol does not reach its full benefit, i.e. a protocol i compatible with a set of protocols P produces a benefit that, for simplicity, we can assume to be proportional to $Q_i e^{N_i} + \sum_{j \in P} Q_j e^{N_j}$. (In other words, we assume that in ‘compatibility mode’ a protocol produces exactly the same benefit as the original protocol it is compatible to.)

In this model, the locking mechanism would not have any effect any more, as long as new protocols are compatible to the existing protocols. Communication protocols could gradually improve and hence raise the overall benefit. However, for two reasons the model, as presented so far, does not apply in practice. Firstly, protocols that are fully compatible to other protocols are difficult to implement and to maintain. (This is even more so, since for economic reasons players will try to avoid others becoming compatible.) Secondly, switching an instance from one protocol to another reduces the achievable gain in benefit

by the *switching costs*. Only if a net benefit remains will a player decide to switch indeed.

Switching costs comprise, e.g., the investment in new licenses, training costs, and the like. They also reflect the risk originating from insufficient compatibility. The latter are crucial since the cost associated with this risk rises with the number of instances that a new protocol is assumed to be compatible with, i.e. with e^{N_j} . As a result, the locking is still in effect and the according entry barrier for a new protocol rises like

$$Q^* = Q \left(1 + R e^N \right) \quad (2)$$

where N is the number of instances of the existing protocol, Q the respective quality, and R describes the compatibility risk: $R = 0$ means no compatibility risk, $R = 1$ no compatibility at all. (We omit the linear switching cost according to licenses, etc.)

From equation (2) we read off again the two cases studied above: $Q^* = Q$ for no switching risk and $Q^* \propto e^N$ for no compatibility. Note that for any $R > 0$, i.e. even for a small compatibility risk, there is a number $N \simeq -\ln R$ where the exponential increase starts to dominate the entry barrier. Solving for R we find

$$R^{-1} \propto e^{\lambda \Delta t} \quad (3)$$

or

$$\Delta t \propto \ln \ln R^{-1} \quad (4)$$

In other words, the compatibility risk rapidly drives the game towards the original game that did not contain compatibility at all.

III. THE EFFECT OF MIDDLEBOXES

Middleboxes extend the network game with a third ingredient that stands outside the effects of both the network lock and the application lock.¹ Thereby, they relieve the network protocol designers from the burden of thorough and rigid standardization, and they can meliorate the locking effects dictated by Metcalf's law. In other words, middleboxes modify the rules of both areas of the game so that the outcome changes thoroughly.

In detail, middleboxes influence the game via two different mechanisms. At first, they transparently mediate between different protocols, freeing users from the need to stick to one solution. Thereby, they consolidate the basis on which Metcalf's law operates. Consider, e.g., a network with N incompatible (application layer) protocols,

¹ Although, our arguments apply to general protocol instances at intermediate positions in a communication network, the argument given here focuses on programmable middleboxes, i.e. active nodes of a programmable network.

each of which is used by n_i network nodes respectively. Without middleboxes the benefit is $\sum_{i=1}^N e^{n_i}$. With the installation of middleboxes the benefit rises to $\exp \sum_{i=1}^N n_i$, the full benefit of mutually compatible protocols.

Secondly, middleboxes provide much more flexibility in their algorithms and protocols. This is the well-known argument of rapid service deployment promoted in the active and programmable networking community. In terms of our analysis, this reduces the switching cost and especially the switching risk for each player in the game. If a service can be quickly deployed and, much more importantly, quickly *withdrawn* when it proves to be incompatible or otherwise harmful, the risk cost are greatly reduced.

Moreover, reducing the number of instances brings down the management overhead and thus the switching costs. This is the well-known effect achieved by the typical middleboxes defined in [1]. Consider, e.g., a firewall. By reducing the potential target of an attacker to one machine, namely the firewall, the attack can be fought more efficiently. Updating security mechanisms on one firewall machine only is much cheaper than securing all the individual hosts in the subnet.

Although the compatibility enforcement achieved by the transparent protocol translation is an extremely valuable effect in practice, this is, in principle, not a new mechanism. Typically, new communication protocols have often been designed to be compatible to established protocols. It is rather the change in the risk-related switching costs that is the key mechanism that makes the difference. This changes the strategic perspectives of the game: Nearly eliminating the compatibility risk reduces the force that drives a given problem space into the lock. As consequence, this lock is much less intense than without the option of middleboxes which in turn reduces the entry-barrier for new players.

As has been shown in the previous section, a small doubt about the compatibility of a new protocol quickly destroys its ability to prevail over an established protocol. If, however, protocols are fully equivalent, i.e. identical, with respect to the end-systems, there is no way to define a risk-based switching cost for the users and the entry barrier vanishes.

It is important to remember that, in order to exploit this mechanism best, it should be completely transparent. If a user needs to activate the mechanism, a small risk is re-introduced into the game, which, if quick withdrawal cannot be assured, spoils the effect. Optimally, the users should not even notice that middleboxes modify the communication protocols they are using. (Except of a better performance or enhanced connectivity and interoperability, of course.) This, at the same time, also means that the

network operators decide what they think to be most suitable for their customers, in extreme, without even asking for their permission. Such paternalism may seem inappropriate and service providers will hence most likely refrain from employing it in this pure form. Nevertheless, the strategy is logically sound even though it might not seem so at first sight.

Given that service providers can be convinced of the strategic benefit of this approach, e.g., because a service improves their networks' efficiency or security, protocol designers and software vendors face a different game. As has been demonstrated, up to now their strategy should be to release a protocol as quickly as possible. As soon as a solution solves a small but sufficient part of a problem, the solution becomes quickly predominant and blocks successors from taking over. Hence, today, the optimal strategy is *time to market*, not quality. Middleboxes, however, weaken the lock and reduce the entry barrier. If users are not aware of the middleboxes that create the transparent protocol translation and if network operators can safely manage the network and, e.g., quickly withdraw a faulty service from the middleboxes, then, the entry barrier can be greatly reduced. As a consequence software vendors are forced to provide *quality* instead of quick but faulty solutions. When the lock is removed, protocol developers have to constantly provide better solutions to remain ahead of the crowd.

IV. CONCLUSION

This paper has given some arguments for the importance of programmable middleboxes. Based on a game theoretic analysis strategic options for the development of communication protocols were derived. They showed that, traditionally, reducing the *time to market* is the optimal choice. It was further shown that middleboxes can overcome this situation by providing transparent compatibility between different protocols. To be fully effective, these middleboxes should be managed by the network providers. It is of utmost importance that, in case of failure, a faulty component (software or hardware) of these middleboxes can be quickly withdrawn without affecting the users. If this can be achieved, the entry barrier for new protocols is removed and *quality* becomes the predominant strategic option for developers, thereby enabling a constant increase in quality. Even more, in game theoretic terms, this changes the equilibrium to which the "protocol development game" is driven, such that, *the mere option of having programmable middleboxes* forces the developers and vendors to change their strategy towards *improving the quality of their protocol designs*.

Concerning the design and development of programmable networking architectures, we can conclude that not only rapid service creation, but also rapid withdrawal of faulty components is of great importance. Moreover, infrastructure services that are managed by the network and service providers, and not directly noticed by the users, can further support this strategy change towards higher quality communication protocols.

REFERENCES

- [1] Brian E. Carpenter and Scott W. Brim. Middleboxes: Taxonomy and issues. RFC 3234, February 2002.
- [2] David D. Clark, John Wroclawski, Karen R. Sollins, and Robert Braden. Tussle in cyberspace: Defining tomorrow's internet. In *Proceedings of the ACM SIGCOMM*, 2002.
- [3] Martin J. Osborne and Ariel Rubinstein. *A Course in Game Theory*. The MIT Press, Cambridge MA, 2001.
- [4] Pyda Srisuresh, Jiri Kuthan, Jonathan Rosenberg, Andrew Molitor, and Abdallah Rayhan. Middlebox communication architecture and framework. RFC 3303, August 2002.
- [5] Richard Swale, Paul Sijben, Philip Mart, Scott Brim, and Melinda Shore. Middlebox communications (midcom) protocol requirements. RFC 3304, August 2002.